

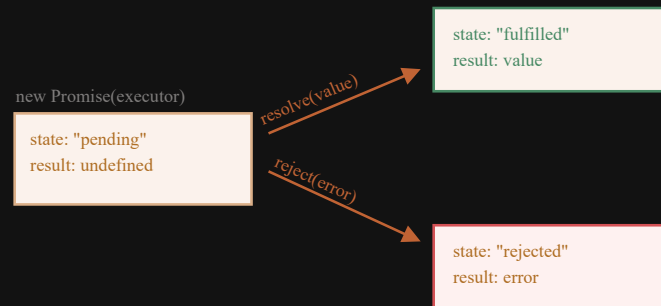
# Promise, promise chaining

Fetch és REST API

# Promise

# Mi a promise?

- Egy olyan objektum, ami "helyettesíti" egy művelet eredményét, amíg az végbe nem megy
- Egy művelet lehet sikeres vagy sikertelen
- Egy proxy ("közvetítő") egy olyan értéknek, amit nem feltétlen tudunk a kód futásakor
- "Ígérem, visszaadom az eredményt"
- Egy promise-nak 3 állapota van:
  - `pending` : alap állapot, még nincs teljesítve
  - `fulfilled` : sikeresen teljesítve
  - `rejected` : teljesítés sikertelen
  - Az utóbbi kettő esetében beszélhetünk arról, hogy a promise *"settled"*
- Ha egyszer egy promise `fulfilled` vagy `rejected` , akkor azt már nem lehet megváltoztatni



# Promise létrehozása

- A `new Promise(executor)` konstruktorral lehet létrehozni
  - Az `executor` egy callback függvény, amit a konstruktor végrehajt
  - A függvénynek kell két paraméter: egy `resolve` és egy `reject`
    - A `resolve(value)` hatására `fulfilled` lesz a promise `value` értékkel
    - A `reject(error)` hatására `rejected` lesz a promise `error` hibával

```
1  new Promise((resolve, reject) => {  
2    if (Math.random() < 0.5) {  
3      resolve("Siker!");  
4    }  
5    reject("Hiba!");  
6  })
```

*50%-50% esély van a sikerre vagy hibára*

# A promise eredménye (consume)

- Az előző executer-ben nincs aszinkron művelet, így rögtön settled lesz
- Ha kiírjuk a konzolra, egy objektumot kapunk vissza, de értékeit nem tudjuk elérni
- A promise eredményének az eléréséhez a `.then()` / `.catch()` / `.finally()` függvényeket használhatjuk
  - `.then()` : ha a promise `fulfilled`
  - `.catch()` : ha a promise `rejected`
  - `.finally()` : független a promise eredményétől, a lényeg, hogy settled

```
1 new Promise((resolve, reject) => {
2   if (Math.random() < 0.5) {
3     resolve("Siker!");
4   }
5   reject("Hiba!");
6 })
7   .then(text => console.log(text));
```

Ez csak a `fulfilled` eseteket kezeli!

# Hibakezelés

- A `.then()` második paraméterében megadható egy függvény a hibakezelésre
- De általában a `.catch()` függvényt preferáljuk
- A `.then()` visszaad egy új promise-t, így hozzá tudjuk láncolni a `.catch()` -t

```
1  new Promise((resolve, reject) => {
2    if (Math.random() < 0.5) {
3      resolve("Siker!");
4    }
5    reject("Hiba!");
6  })
7    .then(text => console.log(text))
8    .catch(error => console.error(error));
```

# Végül

- Akkor fut le, ha a promise bármilyen settled állapotban van
- Hasznos "takarításra" (pl. töltő ikon törlése, nem szükséges kapcsolat megszakítása)

```
1  new Promise((resolve, reject) => {
2    if (Math.random() < 0.5) {
3      resolve("Siker!");
4    }
5    reject("Hiba!");
6  })
7    .then(text => console.log(text))
8    .catch(error => console.error(error))
9    .finally(() => {
10     document.querySelector('#spinner').classList.add('hidden');
11   });
```

# Promise chaining



# Emlékeztető: callback hell

- Egymás után kell különböző időhosszú feladatokat elvégeznünk
- A következő feladatot az előzőnek adjuk meg, mint callback function
- A kódunk nehezen olvasható lesz

```
1  goToSchool(() => {  
2    suffer(() => {  
3      goHome(() => {  
4        walkBlokki(() => {  
5          playWithCirmi(() => {  
6            console.log('Mindennel végeztem!!!');  
7          });  
8        });  
9      });  
10    });  
11  });
```

# Mi a promise chaining?

- A promise akkor lesz settled, ha az aszinkron művelet végbe ment
- Azaz ilyenkor tudom, hogy jöhet a következő művelet végrehajtása
- Ha egy `.then()` -ben visszaadunk egy promise-t, akkor annak az eredményét egy új `.then()` -ben tudjuk kezelni
- A `.then()` -ek így egy láncot alkotnak
- A `.catch()` el fogja kapni a hibát a lánc akármelyik részén fordul elő

```
1  goToSchool()  
2    .then(text => suffer())  
3    .then(text => goHome())  
4    .then(text => walkBlokki())  
5    .then(text => playWithCirmi())  
6    .catch(error => console.error(error))  
7    .finally(text => console.log('Mindennel végeztem!!!'));
```

# Promise.all(array)

- Előfordulhat, hogy egyszerre szeretnénk, ha több promise futna, majd ha azok mind befejeződtek csinálunk valamint
- Például: lekérjük egyszerre az autók, tulajdonos, jelenlegi felhasználó és autókereskedés adatait, majd utána rendereljük az oldalt
- A `Promise.all()` egy iterálható objektumot (pl. tömb) vár argumentumnak, ami promise-okat tartalmaz
- Egy promise-t fog visszaadni, ha mindegyik promise teljesült, aminek értéke egy tömb az összes promise eredményével

```
1  Promise.all([
2    getCars(),
3    getOwner(),
4    getUser(),
5    getDealership(),
6  ])
7    .then(values => console.log(values))
8    .catch(error => console.error(error));
```